

Smallest Positive Initial Value (Greedy + Reverse Thinking)

Pattern: Reverse Simulation | Minimum Initial Resource | Backward DP/Greedy | Binary Search on Answer (Intermediate Step)

Problem

Given an array `arr[]`, find the **smallest initial value x** such that while processing every element from left to right,

If

```
x > arr[i]
```

then

```
x = x + (x-arr[i])
```

Else

```
x = x - (arr[i]-x)
```

At **no point** should x become negative.

Example

```
Input:  
arr = [3,4,3,2,4]
```

```
Output:  
4
```

Step 1 : Think Brute Force

The question asks

Find the smallest x.

The first natural thought is

Try

```
x = 0
x = 1
x = 2
x = 3
...
```

For every x

simulate the process.

If it never becomes negative

return x.

Brute Force Code

```
def survives(start, arr):
    x = start

    for num in arr:
        if x > num:
            x += (x - num)
        else:
            x -= (num - x)

        if x < 0:
            return False

    return True

def smallestX(arr):
    x = 0

    while True:
        if survives(x, arr):
            return x

        x += 1
```

Dry Run

```
arr=[3,4,3,2,4]
```

Try

```
x=0
```

```
0<=3
```

```
0-3=-3
```

```
Fail
```

Try

```
x=1
```

```
1<=3
```

```
1-2=-1
```

```
Fail
```

Try

```
x=2
```

```
2<=3
```

```
1
```

```
1<=4
```

```
-2
```

Fail

Try

x=3

3

3

2

1

0

-4

Fail

Try

x=4

4

5

6

9

16

28

Success.

Answer = 4

Complexity

Suppose answer is A.

0

1

2

...

A

For every value

simulate N elements.

Time = $O(A \times N)$

Impossible if answer is very large.

Step 2 : Simplify the Formula

The biggest trick of the problem.

Instead of trusting the if-else,

expand both equations.

Case 1

$x > arr[i]$

$x = x + (x - arr[i])$

$= 2x - arr[i]$

Case 2

$x \leq arr[i]$

```
x=x-(arr[i]-x)

=2x-arr[i]
```

Both become

```
new_x = 2*x-arr[i]
```

The if-else was only hiding the actual formula.

Important Interview Observation

Whenever you see

```
if(...)
    expression1

else
    expression2
```

Always simplify both.

Sometimes both reduce to exactly the same equation.

This is a common interview trick.

Step 3 : First Optimization (Binary Search)

Now the update is

```
new = 2*x-arr[i]
```

Suppose

```
x=4
```

works.

Will

x=5

work?

Let's compare.

4

↓

$2*4 - a$

5

↓

$2*5 - a$

Difference

$(10 - a) - (8 - a) = 2$

The larger value always stays larger.

Therefore

If 4 works

5 works

6 works

100 works

Answer pattern becomes

FFFFTTTTTT

Whenever answers become

False False False True True True

think

Binary Search on Answer

Binary Search Code

```
def valid(start, arr):  
    x = start  
  
    for num in arr:  
        x = 2*x-num  
  
        if x < 0:  
            return False  
  
    return True  
  
def smallestX(arr):  
    low = 0  
    high = max(arr)  
  
    ans = high  
  
    while low <= high:  
        mid = (low+high)//2  
  
        if valid(mid, arr):  
            ans = mid  
            high = mid-1  
  
        else:  
            low = mid+1  
  
    return ans
```

Complexity

Check

$O(N)$

Binary Search

$O(\log(\max(arr)))$

Total

$O(N \log M)$

where

$M = \max(arr)$

Step 4 : Can We Do Better?

Observe

We are still checking many values.

Can we directly compute the minimum x ?

Yes.

Instead of moving forward,

think backwards.

This is the key observation.

Reverse Thinking

Forward equation

$next = 2 * curr - arr[i]$

Suppose after processing current element,

we need

```
next >= need
```

What should curr be?

```
2*curr - arr[i] >= need  
2*curr >= need + arr[i]  
curr >= (need + arr[i]) / 2
```

Since curr is integer,

```
curr = ceil((need + arr[i]) / 2)
```

This becomes our recurrence.

Building the Algorithm

Initially

After finishing every element,

we only need

```
>= 0
```

So

```
need = 0
```

Now move backwards.

For every element

```
need = ceil((need + arr[i]) / 2)
```

Finally

need itself becomes the answer.

Optimized Code

```
def smallestX(arr):  
    need = 0  
    for i in range(len(arr)-1,-1,-1):  
        need = (need + arr[i] + 1)//2  
    return need
```

Why

```
ceil(x/2)  
  
=  
  
(x+1)//2
```

for integers.

Avoids floating point operations.

Complete Dry Run

```
arr=[3,4,3,2,4]
```

Start

```
need=0
```

Element

4

need

$=\text{ceil}((0+4)/2)$

=2

Element

2

need

$=\text{ceil}((2+2)/2)$

=2

Element

3

need

$=\text{ceil}((2+3)/2)$

=3

Element

4

need

```
=ceil((3+4)/2)
```

```
=4
```

Element

```
3
```

```
need
```

```
=ceil((4+3)/2)
```

```
=4
```

Answer

```
4
```

Why Backward Works

Forward thinking asks

```
What happens after choosing x?
```

Backward thinking asks

```
What minimum value is required BEFORE reaching here?
```

Backward removes guessing completely.

Instead of trying many x,

we directly compute the minimum requirement.

Evolution of Thinking

Question

↓

Need smallest x

↓

Try every x

↓

Brute Force

↓

Too Slow

↓

Simplify formula

↓

$new = 2 * x - arr[i]$

↓

Monotonic?

↓

Yes

↓

Binary Search

↓

Still not optimal

↓

Can I reverse the process?

↓

Yes

↓

Compute minimum requirement backwards

↓

$O(N)$

Pattern Recognition

Whenever you see

Minimum Initial Health

Minimum Energy

Minimum Power

Minimum Fuel

Minimum Coins

Minimum Starting Value

Minimum Resource

Ask

Can I solve it backwards?

Very often,

the backward recurrence becomes much simpler.

Interview Checklist

Whenever a problem asks

Minimum starting value

check

Step 1

Can I simulate?

Yes

↓

Brute Force

Step 2

Is the answer monotonic?

FFFFTTTT

↓

Binary Search on Answer

Step 3

Can I derive the previous state from the next state?

YES

↓

Reverse DP / Reverse Greedy

Common Patterns Learned

Pattern 1

Hidden Formula Simplification

if

else

↓

Both reduce to same equation.

Examples

- Algebraic transformations
 - Piecewise equations
 - Absolute value
 - Distance problems
-

Pattern 2

Binary Search on Answer

Recognition

Find minimum x

Can check in $O(N)$

Working answers become

FFFFTTTT

Pattern 3 ★

Reverse Requirement DP / Greedy

Think

Don't ask

"What happens?"

Ask

"What is the minimum needed before this?"

Common recurrence

```
need = function(next_need, current)
```

Move from

Right

↓

Left

Similar Problems

Problem	Pattern
Dungeon Game (LC 174)	Reverse DP
Minimum Initial Energy to Finish Tasks	Reverse Greedy
Minimum Fuel Problems	Reverse Requirement
Minimum Initial Health	Reverse Thinking
Binary Search on Answer Problems	Intermediate optimization

Real World Analogy

Imagine climbing stairs.

Forward thinking

Start with some money.

Will it be enough?

You keep guessing.

Backward thinking

At the last stair,

minimum money needed = 0.

Now calculate

how much was needed before reaching this stair.

Repeat backwards.

No guessing.

Final Pattern Sheet

Need minimum starting value?

↓

Can simulate?

↓

Yes

↓

Brute Force

↓

Too Slow?

↓

Simplify equations

↓

Monotonic?

↓

Binary Search

↓

Can derive previous state?

↓

YES

↓

Reverse DP / Reverse Greedy

↓

$O(N)$

Complexity Summary

Approach	Time	Space
Brute Force	$O(\text{answer} \times N)$	$O(1)$
Binary Search	$O(N \log M)$	$O(1)$
Reverse Greedy	$O(N)$	$O(1)$



Biggest Lesson From This Problem

This problem teaches **three levels of optimization**:

1. **Brute Force** – Try every starting value.
2. **Binary Search on Answer** – Use monotonicity after simplifying the equation.
3. **Reverse Requirement Thinking** – Stop guessing entirely and compute the minimum required value from the end.

The final $O(N)$ solution is not about a clever trick—it's about changing the direction of thinking. Whenever you see a problem asking for the **minimum initial resource**, always ask:

"Instead of guessing the start, can I calculate the minimum requirement by working backwards?"

This mindset is the key to solving many advanced greedy and dynamic programming interview problems.